ISL-ViT-Tiny: A Compact Factorised Vision Transformer for Isolated-Word Indian Sign Language Recognition, Evaluated on Held-Out Recording Sessions

Yalamanchi Kushal[1], Nethi Kushala Kumar[1], Dhruseek Rishi Menon[1]

[1]Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, India.

Abstract

Word-level recognition of Indian Sign Language (ISL) on a wearable device has two constraints that published ISL work rarely reports together: the model must be small, and the accuracy figure must come from recording conditions the model has not seen. We built ISL-ViT-Tiny, a factorised spatio-temporal Vision Transformer that reads three detected crops per frame (left hand, right hand, face) over 16 frames, adds 21 hand-joint landmarks per hand to the hand tokens, and is trained on INCLUDE (262 words) after masked-image pretraining on iSign. We evaluated it on INCLUDE recording sessions held out as whole blocks of camera capture identifiers, a proxy for session independence rather than a signer-independent test. On the 472 held-out clips of the 50 deployed words, with predictions restricted to those words and six-view test-time augmentation, the released 4-bit model (2,069,700 bytes; 3,894,214 parameters) reached 85.9% mean top-1 accuracy over three seeds (96.4% top-5); answering from all 262 words gave 72.0%. Landmarks added 12.8 points over a pixel-only control trained with the same recipe, and quantisation-aware training recovered most of the 3.1 points that post-training 4-bit rounding lost. One recipe scored 98.5% on a random-video split and 70.1% on held-out sessions. A learned super-resolution front end raised 180p accuracy from 61.0% to about 75.5%, but the adversarial loss gave a reliable gain in only 2 of 9 controlled comparisons. On a second corpus, CISLR, the released model reached only 29.3% on the deployed words, and it has not been validated on new signers or on wearable hardware.

Keywords: Indian Sign Language recognition, Vision Transformer, spatio-temporal modelling, hand landmarks, 4-bit quantisation, evaluation protocol, super-resolution GAN

1 Introduction

Indian Sign Language is the primary language of a large Deaf community in India; the INCLUDE authors cite more than five million users [1]. An automatic recogniser for isolated ISL words could support communication with non-signers, and a camera on a pair of smart glasses is one place such a recogniser could run. That target sets two requirements that pull against each other. The network must fit a memory and compute budget far below that of standard video models, and its reported accuracy must describe what happens when the signer, room and lighting differ from the training recordings.

The second requirement is the harder one to meet with public ISL data. INCLUDE, the main labelled word-level corpus, contains 4,287 videos of 263 words recorded by experienced signers [1]; the copy used here has 4,257 usable clips of 262 words. A word has a median of 15 clips, many of them back-to-back takes of the same signer in the same session. A random split of such data puts near-identical takes on both sides of the train/test boundary, so a high score can reflect recognition of the recording rather than of the sign. The problem is not specific to sign language: Kapoor and Narayanan document train–test leakage across machine-learning-based science [2]. Within sign language, AUTSL reports 95.95% for its baselines on random splits and 62.02% on its user-independent split [3]. Published INCLUDE results of 85.6% (263 words) and 94.5% (INCLUDE-50) [1], and 93.5 to 94.7% for pose-based models [4], were obtained on the dataset's own split; Section 5.3 shows how much such splits can inflate accuracy on this corpus.

Transformer video models [5, 6] and pose-based sign recognisers [4, 7, 8] give strong results on sign data, and recent work targets edge hardware [9, 10]. In the sources we reviewed (Section 2) we did not find an ISL word recogniser that reports, in one place, a hard size budget, a measured quantisation cost, a split that holds out whole recording sessions, and an application tested against the offline evaluation.

The project's objectives were to (i) reach at least 75% top-1 accuracy on 50 deployable words measured on held-out recording sessions; (ii) keep the model near 2 MB; (iii) use an evaluation protocol that does not reward memorising a session; (iv) integrate the model into a working interface; and (v) test a generative adversarial network (GAN) in a position where its contribution can be isolated. This report audits the implementation at commit `ea092f3` of the public repository and documents what was built and measured.

The contributions, stated at the level the evidence supports, are:

1. A task-specific compact recogniser: a factorised spatial-then-temporal Transformer over hand and face crops, with crop position, detection status and normalised

hand-joint features added to the crop tokens (Section 3.5). The components are established; the combination and its sizing to a 2 MB budget are ours.
2. Controlled measurements on one fixed test set: the landmark stream (+12.8 points, paired $p \leq 5 \times 10^{-9}$ in each seed), post-training versus quantisation-aware 4-bit compression, temporal length, crop resolution and training duration (Section 5).
3. A recording-session-aware evaluation built from capture identifiers, with a matched measurement of how much a random split inflates accuracy (about 28 points for one recipe), and a statement of what the proxy does not establish (Sections 3.10 and 7).
4. An integrated application whose prediction route reproduced the offline per-clip outcome for three classifier variants and three low-resolution front ends, on 50 held-out clips each (Section 5.8).
5. A controlled super-resolution study that separates the benefit of a learned upscaler from the benefit of the adversarial loss (Section 5.7).

2 Related work

ISL datasets and recognisers.

INCLUDE [1] is the reference labelled corpus for isolated ISL words; its best model, a pretrained feature extractor and encoder with a trained decoder, reached 85.6% on 263 words and 94.5% on the 50-word subset. CISLR [11] has a vocabulary of about 4,700 words with roughly 1.5 videos per word, so its authors frame recognition as a one-shot task; their prototype baseline reached 16.81% top-1 on CISLR v1.0-a. iSign [12] provides a large collection of ISL video–sentence pairs and benchmark tasks. Recent ISL systems include a CNN–ViT conversational agent reporting 97.5% accuracy under optimised conditions on a domain-specific dataset [13] and a two-handed dynamic ISL recogniser [14]; Damdoo and Kumar survey the field [15].

Video and pose models for sign recognition.

The Vision Transformer [16] and the data-efficient DeiT training recipe [17] made patch-token models practical on modest datasets. For video, ViViT [5] studies variants that factorise spatial and temporal attention, and TimeSformer [6] compares divided space–time attention with joint attention. Sign Language Transformers [18] applied Transformers to continuous recognition and translation. Pose-based recognisers operate on body and hand keypoints: SPOTER [7] normalises hand poses in a local coordinate frame and reports 63.18% on WLASL100; SAM-SLR [8] fuses skeleton, RGB and depth streams; SignBERT [19] pretrains a hand-model-aware representation; OpenHands [4] releases pose-based models for six sign languages and reports 93.5% (SL-GCN) on INCLUDE, rising to 94.7% with self-supervised pretraining. Our model combines the two families: pixels of detected regions, plus hand landmarks from MediaPipe [20, 21] added to the visual tokens rather than processed by a separate graph network.

Compact models and quantisation.

Integer quantisation with simulated rounding during training [22] and the straight-through gradient estimator [23] are the standard basis of quantisation-aware training; Nagel et al. review post-training and quantisation-aware methods [24]. Group-wise scales, which give each block of weights its own scale, appear in Q-BERT [25] and GPTQ [26]. For sign recognition at the edge, TinyMSLR [9] reports fewer than 2.7 M parameters and 7.2 MB on a 20-class isolated-sign benchmark built from two corpora with a signer-independent protocol, and a Kannada Sign Language study [10] deploys quantised LSTM and Transformer models on a smartphone while noting that its split was not signer-independent.

Evaluation protocols.

WLASL [27] collects more than 2,000 words from over 100 signers, and AUTSL [3] publishes a user-independent benchmark split and reports the random-versus-user-independent gap quoted in Section 1. INCLUDE ships without signer identifiers, so a signer-disjoint split cannot be built from its metadata; the protocol in Section 3.10 reconstructs recording sessions from the camera's running clip counter instead.

Super-resolution for recognition.

SRGAN [28], ESRGAN [29] and Real-ESRGAN [30] use adversarial training [31] to produce perceptually sharp upscaling. Blau and Michaeli show that perceptual quality and distortion measures such as PSNR trade off against each other [32]. Whether super-resolution helps a downstream task is an empirical question: Dai et al. examined it for several vision tasks [33], and Haris et al. trained super-resolution with a detection loss [34]. Our study asks the narrower question of whether the adversarial term, rather than learned upscaling alone, helps a frozen sign recogniser.

Table 1 places the closest works beside this project. Numbers are as reported by each paper; they are not comparable with ours unless dataset, vocabulary and split match, and none of them do.

3 Methodology

3.1 Task definition

Each input is a short video containing one isolated sign. The model f_θ outputs logits over the $K = 262$ INCLUDE words it was trained on. The deployed product answers from a fixed subset $\mathcal{D}$ of $|\mathcal{D}| = 50$ words, declared before any run from the word list of the deployed split file (mask50.py:45). The deployed prediction is

$$\hat{y} = \arg\max_{k \in \mathcal{D}} \ \bar{p}_k, \qquad \bar{\mathbf{p}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \operatorname{softmax}\big(f_\theta(x_v)\big), \tag{1}$$

where x_v is view v of the clip and $\mathcal{V}$ is the set of test-time views (Section 3.11). Restricting the argmax to $\mathcal{D}$ is what a 50-word product does with this network; Section 5.1

Table 1 Closest prior work. Results are as reported by each source; protocols differ, so the values are not a ranking.

Work	Data and task	Input	Model	Protocol and reported result	Relation to this project
INCLUDE [1]	ISL, 263 and 50 words	RGB video	pretrained extractor and encoder, trained decoder	dataset split; 85.6% (263), 94.5% (50)	same corpus; we hold out sessions
OpenHands [4]	6 sign languages incl. INCLUDE	pose keypoints	LSTM, BERT, ST-GCN, SL-GCN	INCLUDE: 93.5% (SL-GCN), 94.7% with pretraining	keypoints only; dataset split
CISLR [11]	ISL, about 4,700 words	RGB (I3D features)	prototype one-shot	16.81% top-1 (v1.0-a)	our cross-corpus test set
SPOTER [7]	ASL WLASL, Argentinian LSA64	2D pose	Transformer	WLASL100 63.18%; LSA64 100%	normalised hand pose, as in our features
SAM-SLR [8]	Turkish AUTSL	skeleton, RGB, depth	GCN and CNN ensemble	signer-independent challenge; 98.42% (RGB track)	multimodal fusion; much larger
AUTSL [3]	Turkish, 226 signs	RGB-D	CNN and LSTM	95.95% random vs 62.02% user-independent	split changes the conclusion
TinyMSLR [9]	20 classes from PHOENIX-2014T and CSL	RGB	ConvNeXt-Tiny and Swin, distilled	signer-independent; 99.01% validation; under 2.7 M parameters, 7.2 MB	edge target; smaller task
Kannada SLR [10]	Kannada SL, 33 words	MediaPipe landmarks	LSTM, BiLSTM, Transformer	not signer-independent; 94.19 to 95.71%; 1,098 KB on phone	phone deployment; split leaks
Nedungadi et al. [13]	ISL, domain dataset	RGB	CNN and ViT	97.5% under optimised conditions	ISL application; protocol differs
This work	ISL INCLUDE, 262 trained, 50 deployed	3 crops and hand landmarks	factorised ViT, 4-bit	held-out sessions; 85.9% masked-50, 3 seeds; 2,069,700 B	–

also reports the unrestricted 262-word accuracy so that the effect of the restriction stays visible.

3.2 Data and vocabulary

The classifier is trained only on INCLUDE [1] (CC-BY-4.0): 4,257 clips of 262 words at 1920×1080 pixels and 25 frames per second. Two other corpora enter as unlabelled or test data. iSign [12] supplied 17,896 clips (one per source video, after a hand-detection filter) for self-supervised pretraining, and CISLR v1.5-a [11] supplied 609 clips of 218 shared words as a cross-corpus test. An early download script transcoded part of INCLUDE to 480p and deleted the originals. The loss fell on whole classes, so compression level would have predicted 50 labels; all 1080p sources were restored from Zenodo before any reported experiment (data/refetch_raw.py).

3.3 Region extraction and missing detections

One code path builds the training cache, the test cache and the application input (crops.py:490, reused by predict.py:42). Thirty-two frames are sampled uniformly from the clip, resized to a height of 720 pixels and letterboxed to 1280×720 (crops.py:163). This resize is also where the super-resolution front end is inserted (Section 3.9). MediaPipe Holistic [20] locates the hands and face; a hand box is a square around the 21 hand landmarks enlarged 1.5 times. When Holistic misses a hand, a region around the forearm predicted by the body pose is upscaled to 256 pixels and passed to a dedicated hand landmarker [21] (crops.py:305). If both fail, the nearest real box is carried into the gap and the frame is flagged as interpolated (crops.py:362). Crops are stored at 128 pixels and resized to 64 pixels at model input.

Two defects found during the project shaped this design. An early fallback that built hand boxes directly from pose geometry reported 100% coverage while many crops showed clothing or wall: coverage measured whether a box existed, not whether it was correct, so every crop now carries a provenance code (direct, region rescue, interpolated, missing). Separately, MediaPipe's detectors kept state between videos even in image mode, and re-processing a clip in the same process changed hand presence on 3 to 6% of frames. The detectors are now rebuilt at the start of every video (crops.py:413), the 472 test clips were re-extracted that way into a separate cache, and every headline number uses that cache.

3.4 Landmark features

For each hand h and frame t, MediaPipe returns joints $\mathbf{J}_{t,h} \in \mathbb{R}^{21\times 3}$ in frame-normalised coordinates. The model computes its own invariant features (isl_vit.py:312), so no inference path can compute them differently. Let $\tilde{\mathbf{J}} = \mathbf{J}\,\mathrm{diag}(a,1,1)$ with $a = 16/9$, which undoes the letterbox aspect ratio, and let $\tilde{\mathbf{j}}_0$ be the wrist row. The handshape descriptor is

$$\mathbf{s}_{t,h} = \frac{\mathrm{vec}\left(\tilde{\mathbf{J}}_{t,h} - \mathbf{1}\,\tilde{\mathbf{j}}_{t,h,0}^{\top}\right)}{\max\left(\max_i \|(\tilde{\mathbf{j}}_{t,h,i} - \tilde{\mathbf{j}}_{t,h,0})_{xy}\|_2,\ 10^{-3}\right)} \in \mathbb{R}^{63}, \tag{2}$$

which removes hand position and distance from the camera. Sign location is the wrist relative to the shoulder midpoint $\mathbf{c}_t$, divided by the shoulder width w_t: $\mathbf{u}_{t,h} = (\tilde{\mathbf{j}}_{t,h,0} - \mathbf{c}_t)_{xy}/w_t \in \mathbb{R}^2$. Seven upper-body pose points expressed in the same body frame, with their visibilities, form a 21-dimensional pose vector $\mathbf{r}_t$. If either shoulder has visibility of 0.3 or less, every body-relative quantity is set to zero. The 65-dimensional hand vector $[\mathbf{s}; \mathbf{u}]$ and the pose vector each pass through a two-layer perceptron (Linear–GELU–Linear, 192 units) whose last layer starts at zero, so at the first training step the landmark model computes exactly what the pixel-only model computes.

3.5 Recognition model

Figure 1 shows the network (isl_vit.py:136). The input is $T = 16$ frames $\times$ $S = 3$ streams of 64×64 RGB crops.

Spatial encoder. One encoder with shared weights processes all $T \cdot S = 48$ crops (isl_vit.py:87). A 16×16 patch embedding gives 16 patch tokens plus a class token, which pass through four pre-norm Transformer blocks of width 192 with three heads; the output class token is the crop embedding $\mathbf{e}_{t,s} \in \mathbb{R}^{192}$. Width and head count match DeiT-Tiny [17], so ImageNet and self-supervised weights load without projection.

Token conditioning. Cropping discards where a hand is, yet location distinguishes many signs. Each crop token therefore receives additive terms:

$$\begin{aligned}\mathbf{z}_{t,s} = \mathbf{e}_{t,s} + \mathbf{E}^{\text{str}}_{s} + \mathbf{E}^{\text{time}}_{t} + g(\mathbf{b}_{t,s}) + (1 - d_{t,s})\,\mathbf{m} \\ + \mathbb{I}[s \in \{\text{L}, \text{R}\}]\, q_{t,s}\, \phi_{\text{hand}}([\mathbf{s}; \mathbf{u}]_{t,s}) + \mathbb{I}[s = \text{F}]\, o_t\, \phi_{\text{pose}}(\mathbf{r}_t),\end{aligned} \tag{3}$$

with learned stream and time embeddings $\mathbf{E}$; box geometry $\mathbf{b} = (x_c, y_c, \text{size})$ mapped by a zero-initialised perceptron g; detection indicator d and a learned missing-detection embedding $\mathbf{m}$; hand-present indicator q; pose-valid indicator o; and landmark projections ϕ (Section 3.4). L, R and F denote the left-hand, right-hand and face streams. The landmark terms add 91,008 parameters and no tokens.

Temporal encoder. The 48 conditioned tokens and a class token pass through four more Transformer blocks, and a linear layer maps the final class token to 262 logits. Attention therefore runs over 17 tokens inside each crop and 49 tokens across the clip, instead of the $48 \times 17 = 816$ tokens that joint space–time attention would process. This is the factorised-encoder arrangement studied in ViViT [5].

3.6 Training procedure

The released model was trained on the 2,845 training clips of the session-disjoint split with validation folded in (Section 3.10), for a fixed 1,000 epochs, keeping the final weights (train.py:244; launch scripts run_landmarks.sh and run_lm_s2_resume.sh). The optimiser is AdamW [35] with a peak learning rate of 5×10^{-4}, 10 warm-up epochs, cosine decay to 1%, batch 64 and weight decay 0.05 on weight matrices only; the spatial encoder learns at 0.1 times the base rate. Regularisation uses label smoothing of 0.1 [36], mixup with $\alpha = 0.2$ on half the batches [37], stochastic depth up to 0.1 [38], gradient clipping at 1.0 and an exponential moving average of the weights. Augmentation applies temporal jitter within uniform segments, scale-jittered crop windows,

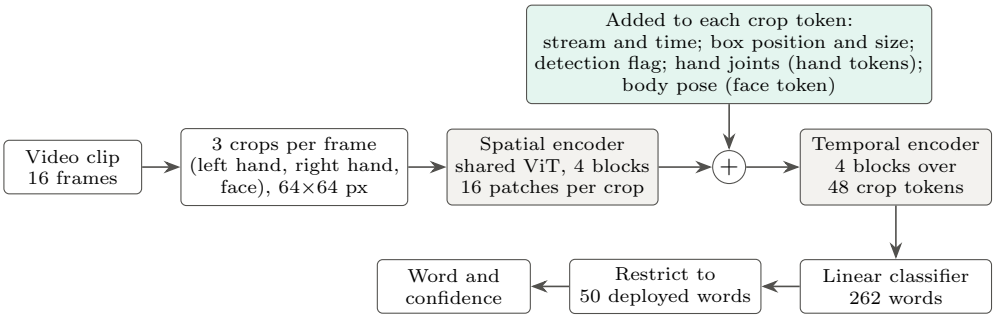


Fig. 1 ISL-ViT-Tiny. Attention is factorised: first within each crop, then across the 48 crop tokens of the clip.

What it shows. The computation graph of the released model. Shaded boxes are Transformer stacks; coloured boxes are the additive token-conditioning terms of Eq. (3). The 50-word restriction is applied at prediction time only (Eq. (1)).

Evidence. Model isl_vit.py:136-299; landmark features isl_vit.py:312-344; masking mask50.py:56-108. Parameter counts were obtained by instantiating the model with report/final/scripts/measure_model.py (new in this report; output report/final/data/model_measurements.json).

horizontal flips that swap the two hand streams and their joints, brightness, contrast and per-channel gain changes, greyscale, stream dropout, and a per-clip resolution jitter. Seeds 0, 1 and 2 give three independent models. The effective recipe of each run is stored in its `summary.json`.

Checkpoint selection on validation was abandoned because validation clips come from the training sessions. On the 262-word split, keeping the final weights instead of the best-validation checkpoint raised test top-1 from 22.1% to 25.3% (Table A1, row E6).

3.7 Self-supervised initialisation

Both encoders are initialised from masked-image pretraining on iSign in the style of SimMIM [39] (pretrain.py:182; launch run_isign_pipeline.sh). A learned mask token replaces 75% of the patch tokens, sampled jointly over time, stream and space; a two-block decoder predicts per-patch-normalised pixels, and the loss is the mean squared error on masked patches. Interpolated crops are excluded from the loss so that the model does not learn to reconstruct stale background. Position and time embeddings are resized when the pretrained geometry differs from the fine-tuning geometry (train.py:75).

3.8 Four-bit quantisation

Every weight matrix (tensors with at least two dimensions) is split into groups G of 128 consecutive weights and quantised symmetrically (export.py:223):

$$\sigma_G = \mathrm{fp16}\Big(\frac{\max_{w\in G}|w|}{7}\Big), \qquad c_w = \mathrm{clamp}\,\big(\mathrm{round}(w/\sigma_G), -8, 7\big), \qquad \hat{w} = \sigma_G\, c_w. \tag{4}$$

Two 4-bit codes are packed per byte with one FP16 scale per group; embeddings, normalisation parameters and biases stay in FP16 (export.py:202, export.py:246). Post-training quantisation (PTQ) applies Eq. (4) to the trained FP32 weights (int4_eval.py). Quantisation-aware training (QAT) continues training for a fixed 300 epochs at a learning rate of 5×10^{-5}, computing the forward pass with $\hat{w}$ and applying the gradients to the full-precision weights, a straight-through estimator [22, 23] implemented by swapping tensors around each step (qat.py:50; launch run_lm_qat.sh). QAT and the exporter call the same target-selection and rounding functions, so the model trained is the model saved. An earlier pixel-only QAT run had chosen its epoch by test accuracy; it was withdrawn, and the landmark QAT was re-run with the final epoch fixed in advance. The release files are written by release.py and read by export.py:181, which rebuilds FP32 weights from the codes. Inference arithmetic is therefore FP32: the 4-bit format reduces storage, not compute.

3.9 Super-resolution front end and GAN study

For video below 720 pixels in height, the frame reader enlarges frames before detection. The default is bicubic interpolation; the alternative is a $\times4$ generator applied at the same point (crops.py:163). The generator is a compact VGG-style network of the kind distributed with the Real-ESRGAN code base, trained on Real-ESRGAN-style synthetic degradations [30]: 3×3 convolutions at low resolution, a final pixel shuffle, and a residual over nearest-neighbour upsampling (sr/models.py:32). Training pairs come from high-resolution hand, face and frame patches of training sessions only, degraded by random blur, resizing, noise and JPEG compression (sr/degrade.py:122). Figure 2 shows the comparison design. The arms are:

- **SR-L1 (no GAN)**: a 621,424-parameter generator trained for 100,000 iterations with pixel L1 loss only.
- **SR-GAN v1**: SR-L1 fine-tuned for 60,000 iterations with L1, VGG19 perceptual [40, 41] and adversarial [31] losses against a spectrally normalised U-Net discriminator [42, 43] (sr/models.py:51). Its comparison with SR-L1 was pre-registered.
- **SR-GAN v2**: v1 trained for 80,000 further iterations with a fresh discriminator; designed after v1's results were seen.
- **SR-GAN v3 and its control**: a 2,039,568-parameter generator with a relativistic average adversarial loss [44], H.264 degradation, and a loss on hand landmarks predicted by a differentiable ResNet-18 [45] heatmap network with soft-argmax readout [46] (sr/train.py:100). The control is the same generator and hand loss without the discriminator. Designed after v1 and v2.

Test inputs are the 472 test clips degraded deterministically inside the frame reader by area downscaling and one JPEG encode: 360p (quality 60), 240p (50) and 180p (40) (sr/degrade.py:35; launch run_gan_study.sh, run_gan_study_v2.sh, run_gan_v3_test.sh). The three released classifiers are scored unchanged on each front end's output. Before the first test run, the rule for a verdict was fixed: "helps" requires a higher mean, all three seeds in the same direction, and pooled exact McNemar $p < 0.05$ (gan_report.py:98).

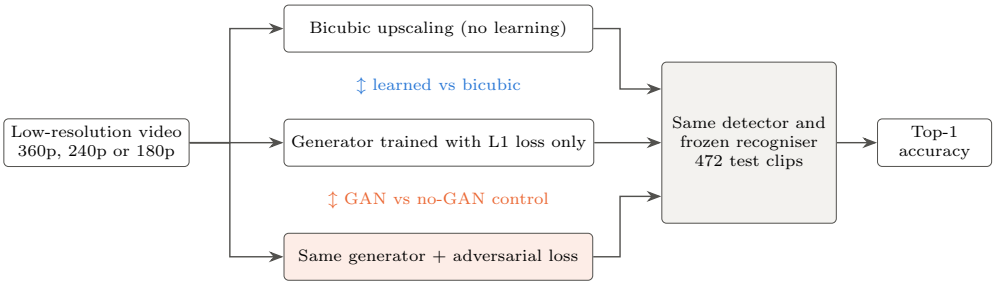


Fig. 2 Design of the super-resolution comparison. Only the upscaling step changes between arms.

What it shows. The two comparisons the study makes. Bicubic versus a learned upscaler measures what learning to upscale buys; a GAN versus the same generator without the discriminator measures what the adversarial term buys. v3 has its own matched control (same generator and hand loss, no discriminator).

Evidence. Networks sr/models.py; training sr/train.py:137; insertion point crops.py:163; drivers run_gan_study.sh, run_gan_v3.sh. Generator weights are local only; their SHA-256 hashes are recorded in each test cache's front-end description, copied into runs/gan_study/detection/*.json.

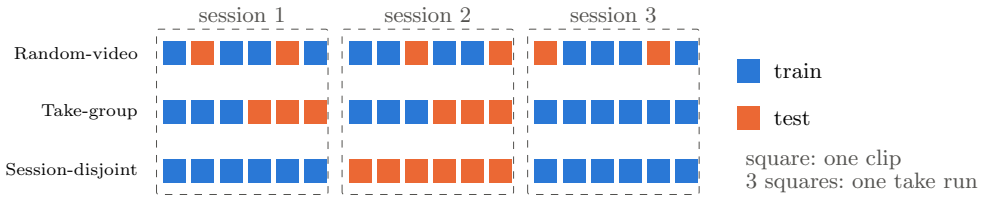


Fig. 3 Three split protocols on the same recordings. Each session holds several words, each recorded as a run of near-identical takes.

What it shows. A schematic, not data. Random-video splits put takes of one run on both sides; take-group splits keep a run together but let one session supply both training and test clips; session-disjoint splits hold out whole sessions.

Measured on the split files. With blocks rebuilt independently from file names (report/final/scripts/audit_results.py), the 262-word random-video split has 666 take-groups on both sides of the train/test boundary; the take-group split has all 13 blocks on both sides; the training split of the final models (full263__session-disjoint-foldval.csv) and the deployed split (vocab50clean__session-disjoint.csv) have none.

What it does not show. Session blocks are inferred from counters, not from signer identities, which INCLUDE does not publish. A held-out block may contain a signer who also appears in training blocks.

3.10 Evaluation protocol

INCLUDE file names carry the camera's running clip counter (`MVI_xxxx`). Clips of one word whose counters differ by at most 5 form a *take-group* of back-to-back recordings (splits.py:118); the gap histogram is bimodal, and thresholds from 5 to 20 give the same partition. Clustering the counters of the whole corpus with the same gap gives 13 contiguous *session blocks* (splits.py:170). The session-disjoint split assigns whole blocks to test by random search over 20,000 candidate assignments, scored mainly on how many test words keep at least six training clips (splits.py:209). Blocks 0, 5 and 6 form the test side. Figure 3 contrasts the three protocols.

For final models, the validation clips (taken from training sessions at take-group granularity) are folded back into training, giving 2,845 training clips over 262 words. The held-out blocks contain 1,412 clips: 1,010 form the test set, and 402 belong to

words with fewer than six training clips and are marked unused. The deployed test set is the 472 test clips of the 50 deployed words (_test472_paths.csv); the audit script confirmed that this list equals the test rows of the deployed split and that every headline result file scores exactly these clips.

3.11 Metrics, test-time views and statistics

Top-1 and top-5 accuracy are the share of clips whose true word is ranked first or within the first five; balanced accuracy is the mean per-word recall (tta.py:65). *Masked-50* applies Eq. (1) with $\mathcal{D}$ the deployed words; *wide-262* takes the argmax over all 262 outputs. Test-time augmentation (TTA) averages softmax probabilities over six views: three temporal phases (0.25, 0.5 and 0.75 of each sampling segment), each with and without a horizontal flip that swaps the hand streams (tta.py:46). Each headline figure is the mean of three seeds; the best seed is never quoted. Paired comparisons on identical clips use the exact two-sided McNemar test [47] on the discordant clips, per seed and pooled over seeds. Five seeds of one earlier configuration gave a top-1 standard deviation of 1.9 points on 1,010 clips (runs/sd263_ssl_foldval*), so differences below about 5 points between single runs were treated as unresolved.

3.12 Application path

The application is a standard-library HTTP server (serve.py:102) and a seven-screen browser interface built into one page from islvit/ui/src. `POST /predict` receives raw video bytes and runs the same extraction, landmark and view functions as the offline evaluation (serve.py:50). The demo launcher (demo.py:69) replaces the server's classifier with one that applies the 50-word restriction exactly as in Eq. (1), and can bind a super-resolution generator into the frame reader. The model is chosen with `--run` at launch; the interface itself has no model selector. A prediction whose confidence is below a threshold τ (default 0.40) goes to a screen listing five candidates, and a user's correction is appended to a log. Figure 4 shows the offline and online paths.

3.13 Software and hardware

Experiments ran on one desktop: NVIDIA GeForce RTX 3060 (12 GB), AMD Ryzen 5 5500, 32 GB RAM, Windows 11, Python 3.11.15, PyTorch 2.6.0 with CUDA 12.4, MediaPipe 1.0.0 (Tasks API), timm 1.0.28 and OpenCV. Training seeds were 0, 1 and 2; split construction used seed 1337.

4 Experimental setup

Table 2 lists the settings of the principal experiments. All results in Sections 5.1 to 5.5 use the same 472 test clips unless the table says otherwise; experiments on other test sets are reported separately and are not ranked against the headline. Appendix A links each experiment to its launch script and result files.

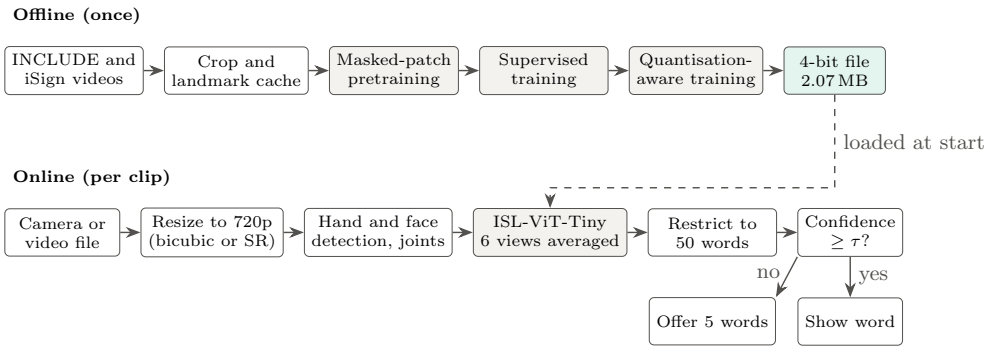


Fig. 4 End-to-end system: offline training and compression, and the per-clip inference path shared by the evaluation and the application.

What it shows. Shaded boxes are learned stages. The online path is shared by the offline test cache and the server: predict.py:42-107, serve.py:50-99, demo.py:69-118. Training commands: run_isign_pipeline.sh (pretraining), run_landmarks.sh (supervised), run_lm_qat.sh (QAT), release.py (release file).

5 Results

5.1 Main result

Observed. The released 4-bit landmark model reached 85.9% top-1 (seeds 84.5, 87.5 and 85.8), 96.4% top-5 and 85.8% balanced accuracy.

How measured. Masked-50 with six-view TTA on the 472 held-out-session test clips; mean of three independently trained seeds; test cache extracted with a freshly built detector per video.

Comparison. Answering from all 262 words gives 72.0% top-1 (69.7, 75.8, 70.3) and 92.7% top-5 on the same clips; a single centre view with the mask gives 85.5%.

Evidence. runs/f16_262w_lm_s*_qat4/masked50_clean.json (keys `masked_tta`, `wide_tta`, `masked_plain`); evaluator mask50.py:56; split vocab50clean_session-disjoint.csv. During this audit we re-scored the three files in release/ on the local test cache; per-clip outcomes matched the tracked files exactly for all three seeds (report/final/audit/rescore_release_s*.json).

Interpretation. The figure describes 50 words of INCLUDE studio video from held-out sessions. It does not describe the 262-word task (64.7% top-1 on all 1,010 test clips, single view, from each QAT run's `summary.json`), other corpora (Section 5.6) or new signers.

5.2 Model variants

The repository contains one architecture family. Its variants differ in input streams, numerical precision, temporal length and preprocessing, not in network type; no CNN, recurrent or 3D-convolutional baseline was trained on the project's split. Table 3 and Fig. 5 compare the four variants that share the test set, the training data and the recipe.

Landmark stream.

Adding the landmark stream, with nothing else changed, raised top-1 from 73.7% to 86.5%. The gain replicated in every seed: 76, 83 and 88 clips gained against 18, 21 and 26 lost ($p = 1.2 \times 10^{-9}$, 6.7×10^{-10} and 4.6×10^{-9}). The pipeline already ran MediaPipe to place crop boxes; the landmark model uses joints that the pixel-only model discarded.

Table 2 Settings of the principal experiments.

Experiment	Models compared	Test set	Seeds	Metric
Main result and variants	pixel FP32; landmark FP32; landmark PTQ INT4; landmark QAT INT4 (released)	472 clips, 50 words, held-out sessions	3 each	masked-50, 6-view TTA
Protocol comparison	one 50-word pixel recipe on random-video and session-disjoint splits	200 and 472 clips (different sets)	1 and 2	top-1, single view
Temporal length	8, 16, 24, 32 frames from one 32-frame cache, 50-word pixel recipe	472 clips	1 (16 frames: 2)	top-1, single view and TTA
Crop resolution	64, 112, 128 pixel input, early 8-frame recipe	INCLUDE-50 take-group, 159 clips	1	top-1
Cross-corpus	INCLUDE-trained models on CISLR	609 CISLR clips	1 to 3	top-1
Super-resolution	bicubic, SR-L1, SR-GAN v1, v2, v3, v3 control; frozen INT4 classifiers	472 clips degraded to 360p, 240p, 180p	3	masked-50, TTA; McNemar
Application route	INT4 landmark, FP32 landmark, FP32 pixel (seed 0); INT4 with bicubic, SR-L1 and SR-GAN v2 at 180p; all through `POST /predict`	50 of the 472 clips (one per word)	1	agreement with offline outcome

Table 3 Controlled variants on the 472 held-out clips (masked-50, 6-view TTA, mean of three seeds). Each row changes one thing relative to the row above it.

Variant	Precision	Parameters	Stored size	Top-1	Top-5	Balanced
Pixels only	FP32	3,803,206	15.2 MB[a]	73.7	93.5	73.3
Pixels + landmarks	FP32	3,894,214	15.6 MB[a]	86.5	96.5	86.5
Landmarks, PTQ	INT4, group 128	3,894,214	as QAT[b]	83.4	95.8	83.3
Landmarks, QAT (released)	INT4, group 128	3,894,214	2,069,700 B	85.9	96.4	85.8

Per-seed top-1: pixels only 75.0/73.7/72.2; landmarks 87.3/86.9/85.4; PTQ 83.1/85.2/82.0; QAT 84.5/87.5/85.8. [a]Parameters × 4 bytes, not a measured file. [b]Same packing as the released file; no PTQ release file was written. 2,069,700 B = 1.974 MiB = 2.07 MB.

Quantisation.

Rounding the FP32 landmark weights to 4 bits after training lost 3.1 points (86.5% to 83.4%); quantisation-aware training reached 85.9%. Pooled over seeds, QAT gained 77 clips and lost 41 relative to PTQ ($p = 0.0012$), although only seed 2 reached $p < 0.05$ on its own. Against FP32, QAT gained 37 clips and lost 45 (pooled $p = 0.44$), so three seeds do not resolve a difference between the released 4-bit model and its FP32 parent; seed 0 alone lost significantly (11 gained, 24 lost, $p = 0.041$).

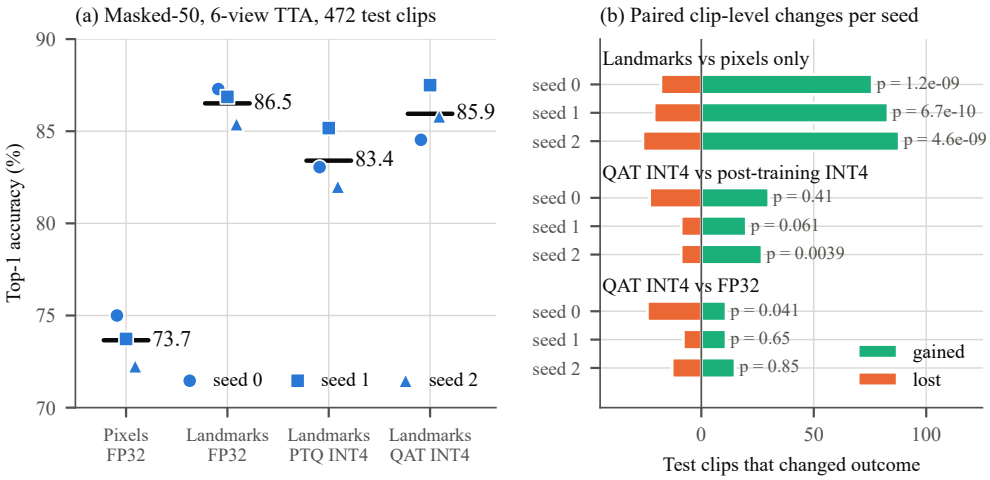


Fig. 5 Controlled variant comparison on the deployed test set.

What it shows. (a) Top-1 accuracy of each seed (markers) and the three-seed mean (bar). (b) For each seed, the test clips that the second model gets right and the first gets wrong (gained) or the reverse (lost), with the exact McNemar p for that seed.

Data. runs/f16_262w_s*/masked50_clean.json, runs/f16_262w_lm_s*/masked50_clean.json, runs/f16_262w_lm_s*/ptq4_clean.json, runs/f16_262w_lm_s*_qat4/masked50_clean.json; keys `masked_tta.top1` and `hits_masked_tta`. Training: run_landmarks.sh, run_lm_qat.sh. Statistics and plot: report/final/scripts/audit_results.py and make_figures.py (new).

Table 4 Compute and runtime of the released model, measured during this audit on the desktop described in Section 3. Nothing was measured on a phone or wearable device.

Quantity	Value	Timing boundary and method
Parameters (landmark model, 262 outputs)	3,894,214	counted on the instantiated network; spatial encoder 1,930,944, temporal blocks 1,779,456, landmark projections 91,008
Multiply–accumulates per view	1.65 G (16 frames); 0.82 G (8 frames)	`torch.utils.flop_counter`; matrix products and convolutions only
Model forward pass, one view, CPU	20.9 ms median (IQR 20.2–21.9 ms)	released seed-0 file, batch 1, FP32 arithmetic after dequantisation, 6 threads, 100 runs after warm-up, idle machine
Server route, one clip, CPU	4.3 s median (10 clips)	`POST /predict` on a 1080p source: decoding, detection and landmarks, 6 views, 50-word mask
Server route, one clip, GPU	3.7 s median (50 clips)	as above, model and detector on the RTX 3060

Sources: report/final/data/model_measurements.json (measure_model.py); report/final/data/app_integration_int4_lm_s0.json and app_integration_int4_lm_s0_cpu.json (app_integration_test.py).

Compute and latency.

Table 4 reports measurements made for this report. Model-only CPU latency was 20.9 ms per view. The end-to-end server route, which decodes a 1080p clip and runs MediaPipe twice over 32 frames, took a median of 4.3 s per clip on the same CPU, so detection and decoding, not the Transformer, dominate the end-to-end time.

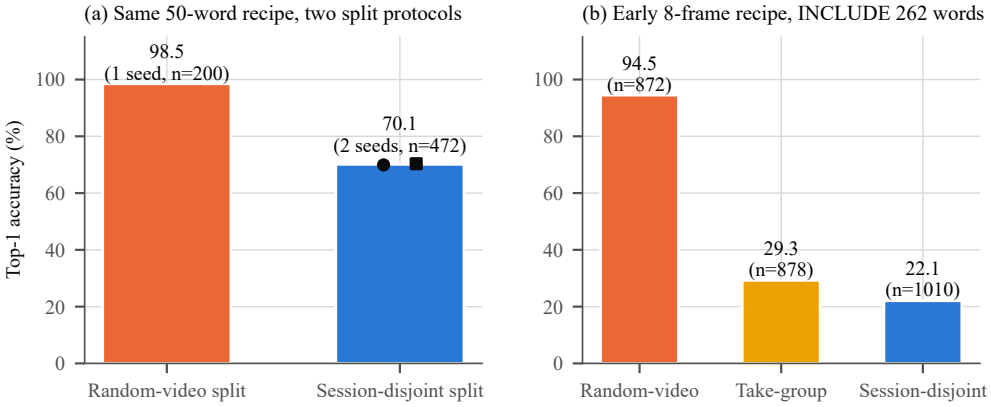


Fig. 6 Accuracy of one recipe under different split protocols.

What it shows. (a) A 50-word pixel-only recipe (16 frames, 2,000 epochs, iSign initialisation) trained on a random-video split and on the session-disjoint split. (b) An early 8-frame, 262-word recipe trained on three split files. Labels give the test-set size n; the bars do not share a test set, because each protocol defines its own.

Data. (a) runs/baseline_random_s0, runs/f16_clean_s0, runs/f16_clean_s1 (key `test.top1`); splits vocab50__random-video.csv and vocab50clean__session-disjoint.csv. (b) runs/full263_v2__random-video, runs/full263_v2__take-group, runs/sd_full263; launch run_phase2.sh.

Interpretation. The random-video score is mostly recognition of the recording; the take-group step removes near-duplicate takes and the session step removes shared recording conditions. The 50-word comparison has one random-split seed and different test clips, so the 28-point gap is an estimate of size, not a paired measurement.

5.3 Effect of the split protocol

The matched 50-word comparison gave 98.5% on the random-video split (one seed, 200 clips) and 70.1% on held-out sessions (69.9% and 70.3%, two seeds, 472 clips), a gap of 28.4 points. Earlier project documents paired the 98.5% with a different 262-word model scored at 75.6%; that pairing understated the gap and has been corrected in the repository (Appendix B). With the early 8-frame recipe on 262 words, accuracy fell from 94.5% (random-video) to 29.3% (take-group) and 22.1% (session-disjoint).

5.4 Temporal length, resolution and training budget

Going from 8 to 16 frames raised single-view top-1 from 60.0% to 70.1% (two-seed mean); 24 and 32 frames gave 69.7% and 72.5% from one seed each, within the noise floor of the 16-frame result. The model uses 16 frames because 32 doubles inference cost without a resolved gain. Larger crops did not help: 64, 112 and 128 pixels gave 46.5%, 46.5% and 44.0% at 0.82, 2.34 and 3.03 GMACs per view (measured with measure_model.py). On the 50-word tier, with a training split that also contained 76 CISLR clips and the same 472 test clips, extending training from 250 to 2,000 epochs raised top-1 by 7.9 points (two runs each; run_longtrain.sh, run_grokking.sh). At 16 frames a 4,000-epoch run gained 0.6 points over 2,000 epochs, below the noise floor. Fine-tuning the 262-word model on the 50-word softmax moved the three seeds in opposite directions (75.6→78.4, 73.5→73.1 and 72.0→68.4) and was rejected. Five further landmark variants (joint augmentation, gap interpolation, velocity, two-hand

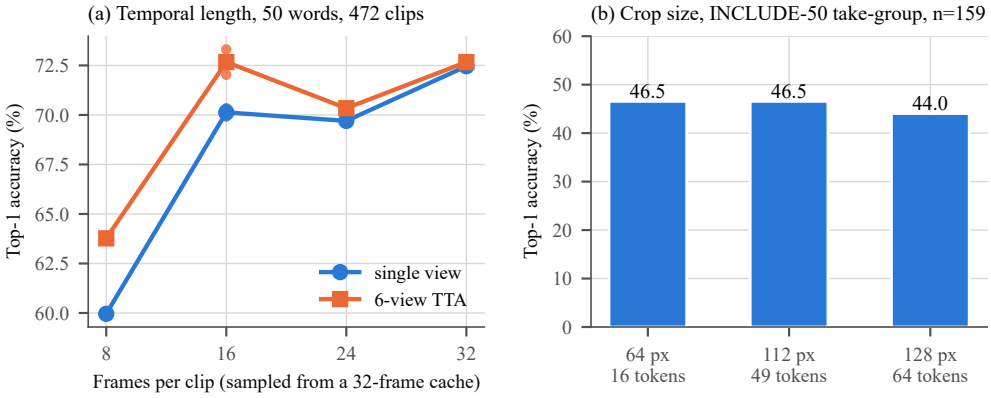


Fig. 7 Temporal length and crop resolution.

What it shows. (a) Top-1 of the 50-word pixel-only recipe with 8, 16, 24 or 32 frames sampled from the same 32-frame cache, single view and six-view TTA; at 16 frames the line is the mean of two seeds (dots). (b) Top-1 at three crop sizes, early 8-frame recipe, INCLUDE-50 take-group split (159 clips, one seed).

Data. (a) runs/f8_clean_s0, f16_clean_s0, f16_clean_s1, f24_clean_s0, f32_clean_s0 (keys `test`, `test_tta`); launch run_frames32.sh. (b) runs/abl64__take-group, abl112__take-group, abl128__take-group; launch run_ablation_res.sh.

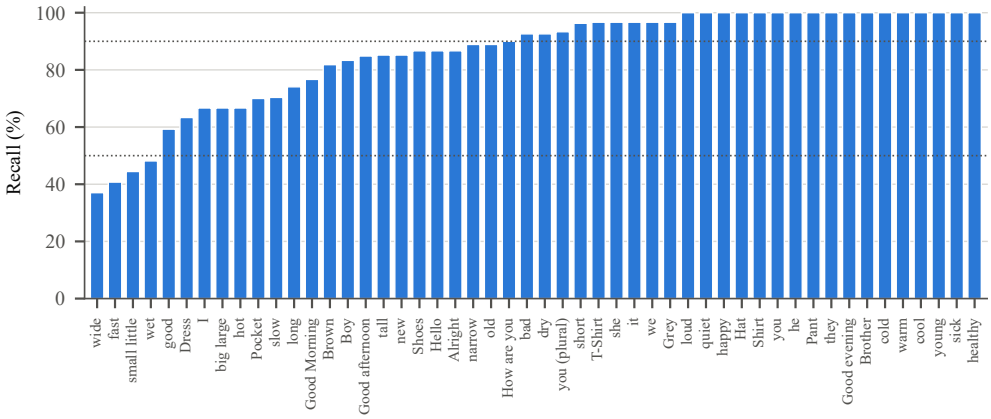


Fig. 8 Per-word recall of the released model on the 472 test clips.

What it shows. Recall per deployed word, pooled over the three released seeds (about 9 to 11 test clips per word per seed), sorted. Dotted lines mark 50% and 90%.

Data. `hits_masked_tta` and `video_paths` in runs/f16_262w_lm_s*_qat4/masked50_clean.json; computed by make_figures.py.

relation and wrist trajectory), screened on 154 validation clips, produced no variant that replicated across seeds, and none was adopted.

5.5 Per-word behaviour

Twenty-seven of the 50 words reached at least 90% recall. Four fell below 50%: *wide* (37.0%), *fast* (40.7%), *small* (44.4%) and *wet* (48.1%). An earlier error analysis on validation clips, recorded in commit `42080f0` rather than in a result file, found that

Table 5 Super-resolution study at 180p (frozen INT4 models, 472 clips, three seeds). Detection is the share of frames with a genuine left-hand detection; on the clean test cache it is 92.0%. Image quality is measured on 1,200 held-out patches.

Front end	GAN loss	Top-1 (%)	Left hand found (%)	PSNR (dB)	SSIM
Bicubic	–	61.0	51.0	30.07	0.856
SR-L1	no	76.0	68.2	31.83	0.900
SR-GAN v1	yes	75.4	67.9	31.07	0.883
SR-GAN v2	yes	75.5	68.6	31.09	0.884
v3 without GAN	no	75.6	65.5	29.96	0.863
SR-GAN v3	yes	75.4	77.6	29.76	0.870

Sources: runs/gan_study/frozen_int4/L3_*.json and runs/gan_study_v1/frozen_int4/L3_*.json; runs/gan_study/detection/L3_*.json and runs/gan_study_v1/detection/L3_srgan.json (`streams.left_hand.genuine`); runs/gan_study/sr_quality*.json and runs/gan_study_v1/sr_quality.json (level `L3 180p`).

about 70% of errors fell in six look-alike groups (for example *big*/*wide*/*narrow* and *good*/*bad*/*wet*) that differ mainly in movement or in the relation between the two hands; the levers aimed at those cues are the ones that failed screening.

5.6 Cross-corpus test

INCLUDE-trained checkpoints from an earlier stage of the project (pixel-only, 8 frames) scored 41.8 to 42.4% on the 1,010 INCLUDE test clips and 0.3 to 1.8% on 609 CISLR clips, where chance is 0.38% (crosscorpus_eval.log; launch run_crosscorpus_eval.sh). Those checkpoints are not the released model. During this audit we extracted landmarks for the CISLR clips and scored the three released files (report/final/scripts/run_crosscorpus_release.sh): answering from all 262 words on the 609 clips, top-1 was 9.6% (seeds 9.5, 8.7, 10.5; top-5 22.4%), and on the 98 CISLR clips of deployed words with the 50-word restriction it was 29.3% (29.6, 27.6, 30.6; chance 2%), against 85.9% on INCLUDE. The landmark model therefore transfers far better than the earlier pixel-only checkpoints, but most of its INCLUDE accuracy does not carry over to another corpus. iSign and CISLR come from the same research group, and we did not check whether any of the unlabelled iSign pretraining videos also appear in CISLR. MediaPipe found a genuine left hand in 25.0% of CISLR frames against 92.0% on the 472 INCLUDE test clips (runs/gan_study/detection/cislr_bicubic.json), so part of the failure lies in detection on CISLR's 300-pixel video.

5.7 Super-resolution and the GAN

Every learned upscaler beat bicubic at every resolution; at 180p the gain was 14.3 to 15.0 points (SR-L1: +15.0, 269 clips gained and 57 lost, $p = 4.5 \times 10^{-34}$). The adversarial loss is a different matter. Of the nine GAN-versus-control comparisons (three GANs at three resolutions), two met the pre-registered "helps" rule, both at 240p: v2 over SR-L1 (+1.5 points, $p = 0.022$) and v3 over its control (+2.4, $p = 0.001$). The second is weaker than it looks, because v3's control was the weakest learned front end at 240p and SR-GAN v3 did not beat SR-L1 there (+0.8, $p = 0.30$). Training v1 longer into v2 lost 1.2 points at 360p ($p = 0.021$). SR-GAN v3 found the most hands at 180p (77.6% of frames against 68.2% for SR-L1), yet its accuracy matched SR-L1's,

Table 6 Integration status of each model variant.

Model	Implemented	Reachable in the app	Tested through the app (this audit)	Benchmarked (472 clips)
Landmark QAT INT4 (released, seed 0)	yes	default `--run`	50 clips: 43 correct, 48 in top five; 50 of 50 outcomes match offline	yes, 3 seeds
Landmark FP32 (seed 0)	yes	--runruns/f16_2 62w_lm_s0	50 clips: 43 correct; 50 of 50 match offline	yes, 3 seeds
Pixels-only FP32 (seed 0)	yes	--runruns/f16_2 62w_s0	50 clips: 35 correct; 50 of 50 match offline	yes, 3 seeds
Landmark PTQ INT4	yes	no release file	not tested	yes, 3 seeds
SR-L1 and SR-GAN v2 front ends	yes	`--sr` generator	50 clips copied at 180p, JPEG 40: bicubic 32, SR-L1 39, SR-GAN v2 37 correct; in each arm 50 of 50 outcomes match the study's seed-0 result file	yes, 3 seeds
SR-GAN v1 and v3 front ends	yes	`--sr` generator	not tested	yes, 3 seeds

Each app test posts raw video to `POST /predict` on the server started by `islvit.demo` and compares the top word with the same model's per-clip outcome in its `masked50_clean.json`. The 50 clips are the first test clip of each deployed word. Script report/final/scripts/app_integration_test.py; outputs report/final/data/app_integration_*.json. Weights under `runs/` are local; only `release/` is public.

so better detection statistics did not translate into better recognition by the frozen classifier. In line with the perception–distortion trade-off [32], the GANs produced sharper patches with lower PSNR than SR-L1.

After the classifiers were fine-tuned for 150 epochs on 360p video through each front end, GAN front ends still beat bicubic at 180p (v1 77.8%, v2 76.9%, bicubic 64.4%). No fine-tuned SR-L1 arm exists, so these results cannot separate the adversarial term from learned upscaling. A 2×2 decomposition at 180p with SR-GAN v2, crossing clean or degraded pixels with clean or degraded landmarks, attributed 7.7 of the 10.5 missing points to degraded landmarks and 2.9 to degraded pixels; the two parts need not sum to the total (runs/gan_study/diagnostics/).

5.8 Application integration and testing

Table 6 separates four levels of evidence for each model: implemented, reachable through the application, tested through the application, and benchmarked on the full test set.

For each of the three classifiers, all 50 server answers had the same correctness as the offline evaluation of that model on that clip, including the clips the model gets wrong. The application therefore runs the evaluated computation: the same detector settings, 16 of 32 frames, six views and the 50-word restriction. The same held for the super-resolution front end: with the 50 clips degraded to 180p (stored losslessly so that no second codec is added; report/final/scripts/make_degraded_clips.py), the server's

answers through bicubic, SR-L1 and SR-GAN v2 matched the study's seed-0 outcomes on every clip, and the ordering on these clips (SR-L1 39, SR-GAN v2 37, bicubic 32 correct) follows the full study. The browser screens were not driven automatically in this audit. Earlier project records describe a Playwright check of all seven screens and live posting of ten demonstration clips, but the harness was not committed, so those outcomes rest on prose alone. The threshold curve on the Settings screen comes from an older pixel-only ensemble (88.7% accuracy at 75.2% coverage for $\tau = 0.4$; runs/gate_mixed.json) and has not been re-measured for the released model.

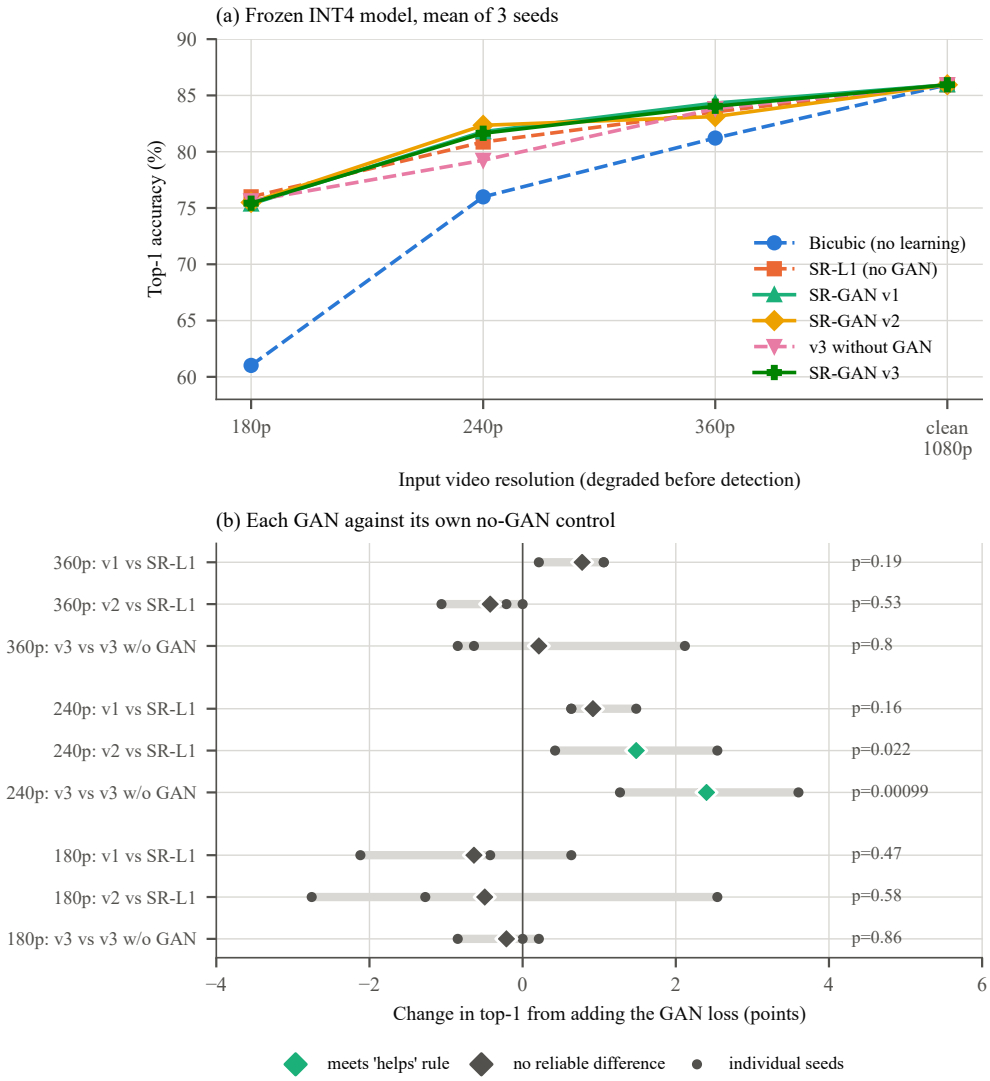


Fig. 9 Learned upscaling versus bicubic, and each GAN versus its own no-GAN control.

What it shows. (a) Mean top-1 of the three frozen released models on the 472 test clips after each front end; dashed lines are arms without an adversarial loss, and clean 1080p is the headline 85.9%. (b) Change in top-1 when the adversarial loss is added to an otherwise identical generator: diamonds are three-seed means, dots individual seeds, and p is the pooled exact McNemar test.

Data. runs/gan_study/frozen_int4/ and runs/gan_study_v1/frozen_int4/ (keys `masked_tta.top1`, `hits_masked_tta`). The original verdicts come from gan_report.py:98; audit_results.py recomputed them with a separate implementation and agreed on every cell.

6 Discussion and originality

What moved accuracy.

The largest single gain came from an input the pipeline already computed: hand joints added 12.8 points for 91,008 parameters. Doubling the frames from 8 to 16 added about 10 points, and training longer helped until 16 frames, then saturated. Larger crops, a narrower training objective and five landmark-derived features did not help. One reading is that with about 11 training clips per deployed word (571 clips for 50 words), explicit handshape measurements give the network information it cannot learn reliably from 64-pixel crops, while extra tokens and pixels mainly add ways to overfit. We did not test this explanation directly.

What compression costs.

Group-wise 4-bit storage reduces the weights from about 15.6 MB to 2.07 MB. Rounding after training cost 3.1 points; training with the rounding in the loop left a gap to FP32 that three seeds do not resolve. The released file is a storage format. The server rebuilds FP32 weights at load time, so the 4-bit model does not yet reduce arithmetic or memory traffic during inference.

What the protocol changes.

A 28-point gap between random-video and session-disjoint splits for one recipe means that INCLUDE accuracies reported on random splits largely measure recognition within recording sessions. Our 85.9% figure sits below published INCLUDE-50 numbers [1, 4] at least partly because it is measured on a harder split; the two sets of numbers cannot be ranked against each other, and ours is not evidence of signer independence.

What the GAN study shows.

Learned upscaling recovered most of the accuracy lost at 180p, and the adversarial term added little on top of it. The 2×2 decomposition suggests a reason: after upscaling, most of the remaining error lies in where the detector places hand joints, which texture realism does not target. The v3 generator, trained against a hand-landmark loss, improved detection statistics but not the frozen classifier's accuracy.

Originality.

We separate three kinds of claim. *Algorithmic novelty*: none is claimed. Factorised video Transformers, masked-image pretraining, MediaPipe landmarks, straight-through quantisation, group-wise scales and GAN super-resolution are all prior work. *System-level originality*: a region-crop factorised Transformer with additive position, detection-status and landmark conditioning, sized and quantised to a 2 MB file, and served by an application that runs the evaluated code path. *Experimental contribution*: seed-replicated, controlled measurements of each design lever on one locked test set; a session-aware split with a measured leakage gap; a cross-corpus drop (85.9% to 29.3%) reported rather than omitted; and a GAN study with a no-GAN control. Table 7 ties each claim to its evidence. Among the works reviewed in Section 2 we

found no ISL word recogniser that reports these properties together; the search was limited to those sources, and the claim should be read in that scope.

7 Limitations and threats to validity

1. **Restricted vocabulary.** The headline restricts predictions to 50 words declared in advance. The same models score 72.0% when allowed to answer from all 262 words on the same clips, and 64.7% on the full 262-word test set.
2. **Session proxy, not signer independence.** Sessions are reconstructed from camera counters because INCLUDE has no signer identifiers. Signer-disjoint testing needs new recordings; a capture protocol exists (docs/CAPTURE_PROTOCOL.md), but no clips have been recorded.
3. **One training corpus.** The headline comes from INCLUDE's studio recordings. On CISLR the released model scored 29.3% (50 deployed words, 98 clips) and 9.6% (262 words, 609 clips), and earlier pixel-only checkpoints were at chance. Nothing here supports a claim of general ISL recognition across cameras, rooms, clothing or signer populations.
4. **Detector dependence.** Accuracy depends on MediaPipe finding hands. Detection falls to 51% of frames at 180p and 25% on CISLR, and the decomposition attributes most of the low-resolution loss to landmark quality.
5. **Small test set.** With 472 clips over 50 words, each word has about 9 to 11 test clips, so one clip moves a word's recall by about 10 points; the three headline seeds span 3 points.
6. **Runtime claims.** Latency was measured on a desktop CPU and GPU only. No phone, glasses or embedded measurement exists, and the 4-bit file is dequantised before inference.
7. **Comparisons not run.** No other architecture family (CNN, recurrent, 3D convolution or pose-only graph network) was trained on the session-disjoint split, so architecture-level comparisons rest on the literature, whose protocols differ. The GAN study lacks a fine-tuned SR-L1 arm, a fine-tuned v3 arm and real low-resolution video.
8. **Reproducibility boundary.** Code, configurations, split files, result files and release models are public. Source videos, caches, training checkpoints and generator weights are local, and the iSign pretraining cache no longer exists, so pretraining would have to be rebuilt. Several statements in older documents (ensemble rows, some detector statistics, the browser test harness) have no committed artifact and are not used as evidence here.

8 Conclusion and future work

The project met its stated targets under the conditions it measured: 85.9% top-1 on 50 deployed words from held-out INCLUDE sessions, in a 2,069,700-byte 4-bit file, served by an application that reproduced the offline per-clip outcomes. Each design choice rests on controlled comparisons on one locked test set, and the comparisons that failed are reported alongside the ones that worked. The work does not establish recognition of new signers, of other corpora or of continuous signing, and it includes

Table 7 Claimed contributions, closest prior approach and evidence.

Contribution	Closest prior approach	Evidence in this project	Qualification
Crops and landmarks in one token stream	pose-only models [4, 7]; multimodal ensembles [8]	+12.8 points over pixels only, 3 seeds, paired $p \leq 5 \times 10^{-9}$ per seed (Fig. 5)	no pose-only baseline trained on our split
2 MB 4-bit model	QAT [22]; group scales [25, 26]	2,069,700 B; PTQ 83.4% vs QAT 85.9%, pooled $p = 0.0012$	storage only; FP32 arithmetic at inference
Session-aware evaluation	user-independent benchmark split [3]	13 blocks, none crossing train/test; 98.5% vs 70.1% for one recipe	sessions inferred from counters, not signers
Evaluated code path in the app	–	300 server answers (three classifiers, three front ends), all matching offline outcomes (Table 6)	50 clips per model; browser screens not automated
GAN with a no-GAN control	task-driven SR [34]; SR for vision tasks [33]	learned SR +14 to +15 points at 180p; GAN term helps in 2 of 9 tests	synthetic degradation; v2 and v3 not pre-registered

no wearable-hardware measurement. Three next steps follow from these gaps: record the planned signer-disjoint set (three signers, 50 words, three takes) and score the released model on it unchanged; train at least one non-Transformer baseline, such as a pose-only graph network, on the same split; and run the model with integer kernels on a phone-class processor, timing the detector and the classifier separately.

Acknowledgements. This work was carried out for the course 23CSE473 Neural Networks and Deep Learning at Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, under the guidance of Dr. T. Bagyammal. Figures elsewhere in the repository that show INCLUDE frames carry the dataset's CC-BY-4.0 attribution.

Declarations

Code availability. All code, configurations, split definitions, result files and the three released models are in the public repository https://github.com/not-good-keeper/compact-vit-sign-language. Links in this report point to commit `ea092f3e25124fcd8929929900e53879d7084943`. Files written for this report are under `report/final/` (scripts, extracted data, figure sources, evidence ledger, discrepancy register and rubric matrix); they are not part of the audited commit.

Data availability. INCLUDE (Zenodo record 4010759), CISLR and iSign are available from their authors. The repository does not redistribute video.

Use of AI assistance. The audit scripts, figures and draft text of this report were prepared with the assistance of an AI model and checked against the repository's result files.

Appendix A Experiment traceability

Appendix B Resolved discrepancies

The full register is `report/final/discrepancy_register.md`. The items that change a number in this report are: (i) the leakage comparison is the matched 98.5% versus 70.1% pair, not 98.5% versus 75.6%; (ii) the headline uses the clean test cache (seed 0: 84.5%), not the `masked50` block inside each QAT run's `summary.json` (seed 0: 85.2%), which came from a cache built with detector state carried between videos; (iii) the earlier pixel-only INT4 figure of 75.6% was selected on test and is not used; (iv) sizes are given in bytes, because older documents wrote "MB" for mebibytes; (v) the earlier cross-corpus numbers belong to pixel-only checkpoints, not to the released model; (vi) the clean-video detection reference for the 472 test clips is 92.0% (left hand), not the 91.1% quoted for the whole corpus; and (vii) the page ranges of several references follow Crossref.

Table A1 Experiment traceability. Links are pinned to commit `ea092f3`.

ID	Question	Controlled change	Launch	Raw result	Shown in
E1	Landmark gain	landmark stream on or off	run_landmarks.sh	f16_262w_s*/masked50_clean.json, f16_262w_lm_s*/masked50_clean.json	Tab. 3, Fig. 5
E2	4-bit cost	PTQ or QAT	run_lm_qat.sh	f16_262w_lm_s*/ptq4_clean.json, f16_262w_lm_s*_qat4/masked50_clean.json	Tab. 3
E3	Clean test cache	fresh detector per video	run_clean_test.sh	masked50_fresh.json vs `_clean`	Sec. 3.3
E4	Leakage size	split protocol	run_frames32.sh	baseline_random_s0, f16_clean_s0, f16_clean_s1	Fig. 6a
E5	Protocol ladder	split protocol	run_phase2.sh	full263_v2__random-video, full263_v2__take-group, sd_full263	Fig. 6b
E6	Checkpoint choice, folding	last vs best-val; validation folded	run_iter3.sh	sd263_lastsel, sd263_foldval	Sec. 3.6
E7	Noise floor	seed only (5 seeds)	run_lever2.sh	sd263_ssl_foldval, `_s1` to `_s4`	Sec. 3.11
E8	Temporal length	frames drawn	run_frames32.sh	f8_clean_s0, f24_clean_s0, f32_clean_s0	Fig. 7a
E9	Crop size	input pixels	run_ablation_res.sh	abl64__take-group, abl112__take-group, abl128__take-group	Fig. 7b
E10	Training length	epochs	run_longtrain.sh, run_push75.sh	long2000_s1, grok_wd005, f16_long4k_s0	Sec. 5.4
E11	Narrow fine-tune	50-word softmax	run_narrow.sh	f16_262w_s*_narrow	Sec. 5.4
E12	Landmark levers	one flag each	run_dev_screen.sh	dev_lm_*/val50.json	Sec. 5.4
E13	Cross-corpus, early models	corpus	run_crosscorpus_eval.sh	crosscorpus_eval.log (log only)	Sec. 5.6
E14	Super-resolution	front end	run_gan_study.sh, run_gan_v3_test.sh	runs/gan_study/, runs/gan_study_v1/	Fig. 9, Tab. 5
E15	App route	model behind the server	app_integration_test.py*	report/final/data/app_integration_*.json*	Tab. 6
E16	Released model on CISLR	corpus	run_crosscorpus_release.sh*	report/final/data/crosscorpus_release_s*.json*	Sec. 5.6

*New in this report (`report/final/`); not in the audited commit. Evaluator for E1 to E3 and E14: mask50.py; for E4 to E11: train.py, which scores the test split at the end of training.

References

[1] Sridhar, A., Ganesan, R.G., Kumar, P., Khapra, M.: INCLUDE: A large scale dataset for Indian sign language recognition. In: Proceedings of the 28th ACM International Conference on Multimedia, pp. 1366–1375 (2020). https://doi.org/10.1145/3394171.3413528

[2] Kapoor, S., Narayanan, A.: Leakage and the reproducibility crisis in machine-learning-based science. Patterns **4**(9), 100804 (2023) https://doi.org/10.1016/j.patter.2023.100804

[3] Sincan, O.M., Keles, H.Y.: AUTSL: A large scale multi-modal Turkish sign language dataset and baseline methods. IEEE Access **8**, 181340–181355 (2020) https://doi.org/10.1109/ACCESS.2020.3028072

[4] Selvaraj, P., NC, G., Kumar, P., Khapra, M.: OpenHands: Making sign language recognition accessible with pose-based pretrained models across languages. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 2114–2133 (2022). https://doi.org/10.18653/v1/2022.acl-long.150

[5] Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M., Schmid, C.: ViViT: A video vision transformer. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6816–6826 (2021). https://doi.org/10.1109/ICCV48922.2021.00676

[6] Bertasius, G., Wang, H., Torresani, L.: Is space-time attention all you need for video understanding? In: Proceedings of the 38th International Conference on Machine Learning. PMLR, vol. 139 (2021). https://arxiv.org/abs/2102.05095

[7] Boháček, M., Hrúz, M.: Sign pose-based transformer for word-level sign language recognition. In: 2022 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW), pp. 182–191 (2022). https://doi.org/10.1109/WACVW54805.2022.00024

[8] Jiang, S., Sun, B., Wang, L., Bai, Y., Li, K., Fu, Y.: Skeleton aware multi-modal sign language recognition. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 3408–3418 (2021). https://doi.org/10.1109/CVPRW53098.2021.00380

[9] Lamaakal, I., Yahyati, C., Maleh, Y., El Makkaoui, K., Ouahbi, I.: An explainable hybrid CNN–transformer model for sign language recognition on edge devices using adaptive fusion and knowledge distillation. Scientific Reports **16**, 7143 (2026) https://doi.org/10.1038/s41598-026-38478-8

[10] Venkatappa, U., K S, N., K S, N., Bhat, P.P., Srinivas, P.: Dynamic Kannada sign language recognition on resource constrained devices. Scientific Reports **16**,

11186 (2026) https://doi.org/10.1038/s41598-026-40181-7

[11] Joshi, A., Bhat, A., S, P., Gole, P., Gupta, S., Agarwal, S., Modi, A.: CISLR: Corpus for Indian sign language recognition. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, pp. 10357–10366 (2022). https://doi.org/10.18653/v1/2022.emnlp-main.707

[12] Joshi, A., Mohanty, R., Kanakanti, M., Mangla, A., Choudhary, S., Barbate, M., Modi, A.: iSign: A benchmark for Indian sign language processing. In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 10827–10844 (2024). https://doi.org/10.18653/v1/2024.findings-acl.643

[13] Nedungadi, P., Dileep, G., Geetha, M., Raman, R.: Vision transformer-powered conversational agent for real-time Indian sign language e-governance accessibility. Scientific Reports **15**, 33055 (2025) https://doi.org/10.1038/s41598-025-12667-3

[14] Govindharajalu Kaliyaperumal, V., Gopalan, P.A.: A deep neural network framework for dynamic two-handed Indian sign language recognition in hearing and speech-impaired communities. Sensors **25**(12), 3652 (2025) https://doi.org/10.3390/s25123652

[15] Damdoo, R., Kumar, P.: An integrative survey on Indian sign language recognition and translation. IET Image Processing **19**(1), 70000 (2025) https://doi.org/10.1049/ipr2.70000

[16] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16x16 words: Transformers for image recognition at scale. In: International Conference on Learning Representations (ICLR) (2021). https://arxiv.org/abs/2010.11929

[17] Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., Jégou, H.: Training data-efficient image transformers & distillation through attention. In: Proceedings of the 38th International Conference on Machine Learning. PMLR, vol. 139, pp. 10347–10357 (2021). https://arxiv.org/abs/2012.12877

[18] Camgoz, N.C., Koller, O., Hadfield, S., Bowden, R.: Sign language transformers: Joint end-to-end sign language recognition and translation. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10020–10030 (2020). https://doi.org/10.1109/CVPR42600.2020.01004

[19] Hu, H., Zhao, W., Zhou, W., Wang, Y., Li, H.: SignBERT: Pre-training of hand-model-aware representation for sign language recognition. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 11067–11076 (2021). https://doi.org/10.1109/ICCV48922.2021.01090

[20] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang,

F., Chang, C.-L., Yong, M.G., Lee, J., Chang, W.-T., Hua, W., Georg, M., Grundmann, M.: MediaPipe: A Framework for Building Perception Pipelines (2019). https://arxiv.org/abs/1906.08172

[21] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., Grundmann, M.: MediaPipe Hands: On-device Real-time Hand Tracking. CVPR Workshop on Computer Vision for Augmented and Virtual Reality (2020). https://arxiv.org/abs/2006.10214

[22] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2704–2713 (2018). https://doi.org/10.1109/CVPR.2018.00286

[23] Bengio, Y., Léonard, N., Courville, A.: Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation (2013). https://arxiv.org/abs/1308.3432

[24] Nagel, M., Fournarakis, M., Amjad, R.A., Bondarenko, Y., Baalen, M., Blankevoort, T.: A White Paper on Neural Network Quantization (2021). https://arxiv.org/abs/2106.08295

[25] Shen, S., Dong, Z., Ye, J., Ma, L., Yao, Z., Gholami, A., Mahoney, M.W., Keutzer, K.: Q-BERT: Hessian based ultra low precision quantization of BERT. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 8815–8821 (2020). https://doi.org/10.1609/aaai.v34i05.6409

[26] Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D.: GPTQ: Accurate post-training quantization for generative pre-trained transformers. In: International Conference on Learning Representations (ICLR) (2023). https://arxiv.org/abs/2210.17323

[27] Li, D., Rodriguez Opazo, C., Yu, X., Li, H.: Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison. In: 2020 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 1448–1458 (2020). https://doi.org/10.1109/WACV45572.2020.9093512

[28] Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., Aitken, A., Tejani, A., Totz, J., Wang, Z., Shi, W.: Photo-realistic single image super-resolution using a generative adversarial network. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 105–114 (2017). https://doi.org/10.1109/CVPR.2017.19

[29] Wang, X., Yu, K., Wu, S., Gu, J., Liu, Y., Dong, C., Qiao, Y., Loy, C.C.: ESRGAN: Enhanced super-resolution generative adversarial networks. In: Computer Vision – ECCV 2018 Workshops. Lecture Notes in Computer Science, pp. 63–79

(2019). https://doi.org/10.1007/978-3-030-11021-5_5

[30] Wang, X., Xie, L., Dong, C., Shan, Y.: Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data. In: 2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), pp. 1905–1914 (2021). https://doi.org/10.1109/ICCVW54120.2021.00217

[31] Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: Advances in Neural Information Processing Systems 27 (2014). https://arxiv.org/abs/1406.2661

[32] Blau, Y., Michaeli, T.: The perception-distortion tradeoff. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 6228–6237 (2018). https://doi.org/10.1109/CVPR.2018.00652

[33] Dai, D., Wang, Y., Chen, Y., Van Gool, L.: Is image super-resolution helpful for other vision tasks? In: 2016 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 1–9 (2016). https://doi.org/10.1109/WACV.2016.7477613

[34] Haris, M., Shakhnarovich, G., Ukita, N.: Task-Driven Super Resolution: Object Detection in Low-resolution Images (2018). https://arxiv.org/abs/1803.11316

[35] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2019). https://arxiv.org/abs/1711.05101

[36] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2818–2826 (2016). https://doi.org/10.1109/CVPR.2016.308

[37] Zhang, H., Cisse, M., Dauphin, Y.N., Lopez-Paz, D.: mixup: Beyond empirical risk minimization. In: International Conference on Learning Representations (ICLR) (2018). https://arxiv.org/abs/1710.09412

[38] Huang, G., Sun, Y., Liu, Z., Sedra, D., Weinberger, K.Q.: Deep networks with stochastic depth. In: Computer Vision – ECCV 2016. Lecture Notes in Computer Science, pp. 646–661 (2016). https://doi.org/10.1007/978-3-319-46493-0_39

[39] Xie, Z., Zhang, Z., Cao, Y., Lin, Y., Bao, J., Yao, Z., Dai, Q., Hu, H.: SimMIM: a simple framework for masked image modeling. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9643–9653 (2022). https://doi.org/10.1109/CVPR52688.2022.00943

[40] Johnson, J., Alahi, A., Fei-Fei, L.: Perceptual losses for real-time style transfer and super-resolution. In: Computer Vision – ECCV 2016. Lecture Notes in Computer Science, pp. 694–711 (2016). https://doi.org/10.1007/978-3-319-46475-6_43

[41] Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. In: International Conference on Learning Representations (ICLR) (2015). https://arxiv.org/abs/1409.1556

[42] Schönfeld, E., Schiele, B., Khoreva, A.: A U-Net based discriminator for generative adversarial networks. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8204–8213 (2020). https://doi.org/10.1109/CVPR42600.2020.00823

[43] Miyato, T., Kataoka, T., Koyama, M., Yoshida, Y.: Spectral normalization for generative adversarial networks. In: International Conference on Learning Representations (ICLR) (2018). https://arxiv.org/abs/1802.05957

[44] Jolicoeur-Martineau, A.: The relativistic discriminator: a key element missing from standard GAN. In: International Conference on Learning Representations (ICLR) (2019). https://arxiv.org/abs/1807.00734

[45] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016). https://doi.org/10.1109/CVPR.2016.90

[46] Sun, X., Xiao, B., Wei, F., Liang, S., Wei, Y.: Integral human pose regression. In: Computer Vision – ECCV 2018. Lecture Notes in Computer Science, pp. 536–553 (2018). https://doi.org/10.1007/978-3-030-01231-1_33

[47] McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947) https://doi.org/10.1007/BF02295996